

Informe — TP 1: Redes Neuronales Feed Forward y Convolucionales

Aprendizaje Automático II — Tecnicatura Universitaria en Inteligencia Artificial, FCEIA - UNR.

Grupo: 9

Integrantes: Nicola Petetta, Francisco Domingo

Este informe cubre los cuatro problemas del trabajo práctico. Cada sección tiene la misma estructura: procedimiento, decisiones de diseño y justificación, desafíos encontrados y conclusiones.

Problema	Tipo	Notebook	Resultado principal en test
1. Riesgo de ACV	Clasificación binaria (MLP)	problema1.ipynb	AUC-ROC 0.8177
2. Dígitos MNIST	Clasificación multiclase (MLP)	problema2.ipynb	Accuracy 0.9802
3. Profit de Superstore	Regresión (MLP)	problema3.ipynb	MAE 17.08 dólares
4. Razas de perros	Clasificación de imágenes (CNN desde cero)	problema4.ipynb	Exactitud 0.825

Los cuatro notebooks corren en Google Colab con GPU. Los notebooks 1 a 3 corren también en CPU, con las rutas locales. Todos fijan sus semillas, y cada notebook verifica que sus resultados se repiten.

Problema 1: Clasificación Binaria (ACV)

Procedimiento

El problema se resolvió en `problema1.ipynb`, en este orden:

1. Análisis exploratorio: distribución del target, histogramas, boxplots, tasa de ACV por categoría y correlación con `stroke`.
2. Preprocesamiento: eliminar `id` y la fila `gender = Other`, dividir en train, validación y test (70/15/15, estratificado), imputar `bmi`, codificar las categóricas y escalar las numéricas. Cada transformación se ajusta solo con train.
3. Diseño del MLP en PyTorch (18 -> 64 -> 32 -> 16 -> 1) con `BCEWithLogitsLoss` y `pos_weight`.
4. Entrenamiento con Adam y Early Stopping, y curvas de aprendizaje.
5. Elección del umbral sobre validación y evaluación en test, una sola vez.
6. Matriz de confusión y análisis del impacto de los falsos negativos.
7. Ablación sobre validación y conclusiones.

Decisiones de diseño y justificación

Datos y preprocesamiento

El dataset tiene 5110 pacientes y un desbalance fuerte de clases. Hay 249 pacientes con `stroke = 1` (4.87%) y 4861 con `stroke = 0` (95.13%), una razón de 19.5 a 1. `age` es el predictor con la correlación más alta con `stroke` (0.245). `bmi` muestra una correlación baja (0.042) y tiene 201 valores faltantes.

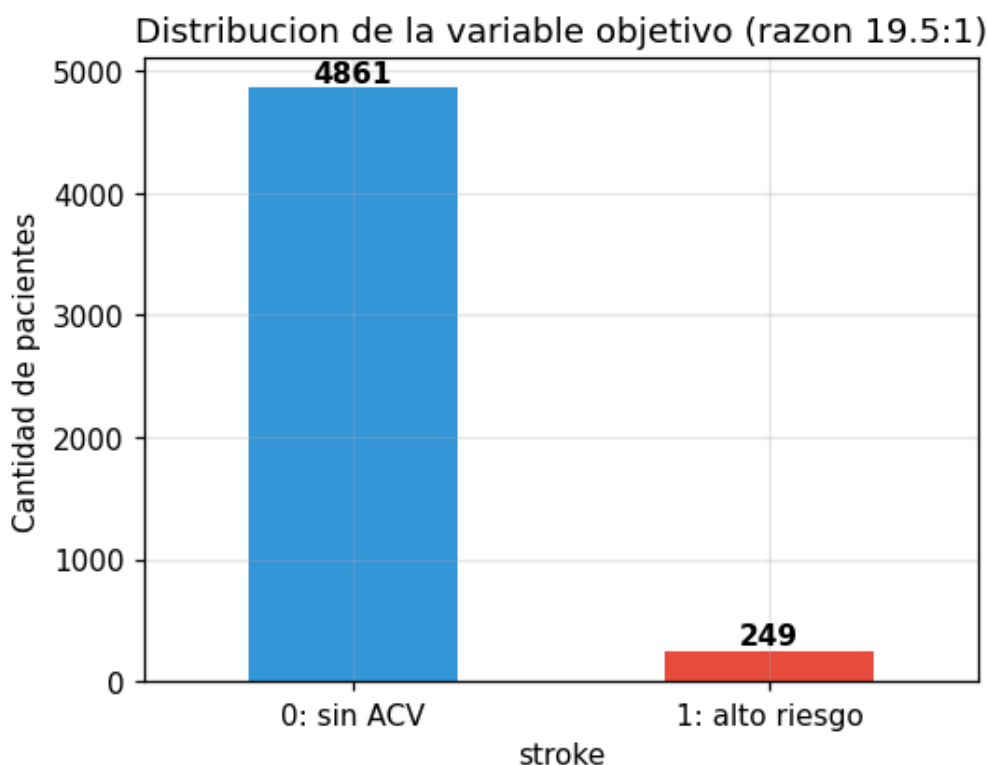


Figura 1. Distribución de la variable objetivo `stroke`. Muestra el desbalance de clases.

El dataset se dividió en train, val y test antes de ajustar cualquier transformación. La mediana de imputación y los parámetros de escalado se calcularon solo con train. Esto evita el *data leakage*. Se imputó `bmi` con la mediana, más robusta que la media frente a la cola larga de valores altos. Se codificaron `work_type`, `Residence_type` y `smoking_status` con one-hot encoding, para no imponer un orden falso entre categorías. Se estandarizaron

`age`, `avg_glucose_level` y `bmi` con z-score. Se eligió z-score en lugar de min-max porque los outliers comprimen el rango del min-max. El detalle está en el notebook, sección de preprocesamiento.

Estrategia frente al desbalance de clases (19.5 : 1). Se compararon tres estrategias. El submuestreo deja menos de 500 filas de entrenamiento, una pérdida grande de datos. SMOTE interpola todas las columnas por igual. Así genera valores como 0.37 en una columna que solo admite 0 o 1. SMOTENC evita ese problema, pero agrega la dependencia `imbalanced-learn` solo para este paso. El sobremuestreo a mano no agrega información nueva: repite las mismas 174 filas positivas de train.

Se eligió **class weighting**. La función de costo penaliza más el error sobre la clase positiva (parámetro `pos_weight` de `BCEWithLogitsLoss`). Entrena sobre los datos originales, sin duplicar filas ni generar datos sintéticos. La motivación fue clínica: un falso negativo es más costoso que un falso positivo.

Corrección posterior, con medición: la ablación muestra que el class weighting cambia las probabilidades de la red. Pero no mejora la detección cuando el umbral se elige sobre validación. Sin `pos_weight`, la precisión con recall ≥ 0.70 es 0.177, igual a la de 0.178 con `pos_weight`. La razón clínica sigue en pie, pero se resuelve mejor eligiendo el umbral.

Arquitectura del modelo

MLP de 3 capas ocultas, 64 -> 32 -> 16 neuronas, 3841 parámetros entrenables en total. El problema tiene 18 features de entrada y 3576 filas de entrenamiento, con solo 174 positivas. Por eso se eligió una red chica.

Activación oculta: Leaky ReLU (`negative_slope = 0.01`), no ReLU. Con ReLU común, una neurona queda "muerta" si su suma ponderada es negativa para todos los ejemplos. Ahí el gradiente es exactamente 0. Leaky ReLU deja un gradiente chico (0.01) en la zona negativa. Es una precaución que sigue el apunte de la Unidad 1. La elección es preventiva, no demostrada.

Activación de salida: ninguna (un logit), no sigmoid ni softmax. Se usa `BCEWithLogitsLoss`, que aplica la sigmoid de forma interna. Es más estable numéricamente que aplicar sigmoid y después `BCELoss`. Softmax con 2 salidas equivale a sigmoid con 1 salida, pero con un parámetro redundante. Además, no tiene un equivalente directo de `pos_weight`.

Función de costo y regularización. `BCEWithLogitsLoss` con `pos_weight = 19.55`. Dropout (0.3) entre las capas ocultas, combinado con Early Stopping sobre la pérdida de validación.

Inicialización de pesos: Kaiming/He explícita. Xavier/Glorot supone que la activación no cambia mucho la escala de la señal. ReLU anula cerca de la mitad de los valores, y eso reduce la varianza a la mitad en cada capa. Kaiming corrige eso con $\text{Var}(w) = 2 / n_{\text{in}}$. Se declaró de forma explícita (`nn.init.kaiming_normal_, nonlinearity='leaky_relu'`), para fijar el `nonlinearity` correcto y documentar la decisión.

Optimizador: Adam, tasa de aprendizaje $1e-3$. Ajusta la tasa de cada parámetro de forma individual.

Entrenamiento y evaluación

Entrenamiento. Máximo 100 épocas, Early Stopping con paciencia 10. El entrenamiento paró en la época 20 y el modelo conservó los pesos de la época 10 (`val_loss` 0.9726, `val_auc` 0.8331). Las curvas muestran sobreajuste leve. En la mejor época, la brecha de pérdida (`val - train`) es 0.1011. Se eligió parar por pérdida porque es más estable que el AUC con solo 37 positivos en validación.

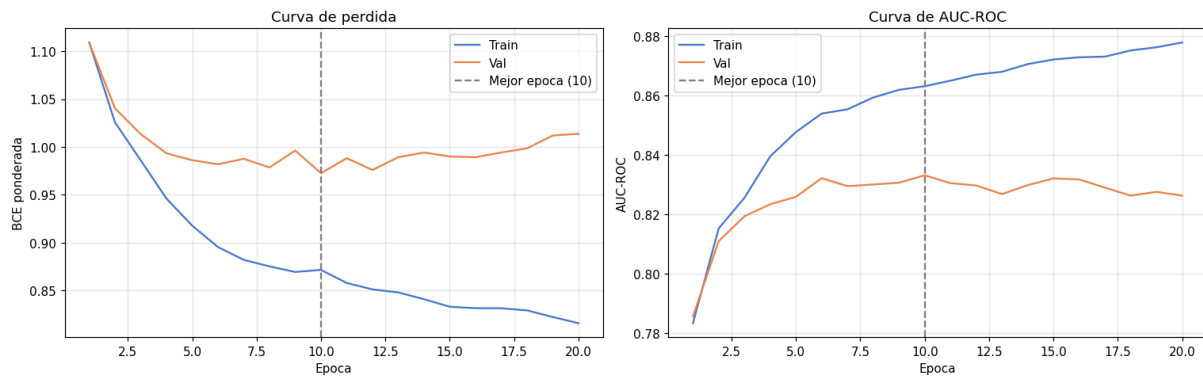


Figura 2. Curvas de aprendizaje de train y validación.

Umbral de decisión. El umbral 0.5 no es neutral: `pos_weight = 19.55` empuja a la red a dar probabilidades altas. Se definió una regla sobre validación: máxima precisión entre los umbrales con recall mayor o igual a 0.70. La regla elige 0.51 (precisión 0.138, recall 0.811), casi igual a 0.50. La elección depende del costo relativo de un falso negativo y de una falsa alarma. Es una decisión clínica, no técnica.

Resultados en test (una sola evaluación, 38 positivos).

Umbral	Accuracy	Precision	Recall	F1	AUC-ROC
0.51 (elegido)	0.7001	0.1160	0.7632	0.2014	0.8177
0.50	0.6923	0.1163	0.7895	0.2027	0.8177
Siempre "sin ACV"	0.9505	0.0000	0.0000	0.0000	0.5000

Lectura. La accuracy engaña: la regla trivial supera al modelo en accuracy y no detecta ningún caso. AUC-ROC es la mejor métrica para comparar modelos, porque no depende del umbral. Recall y precisión describen el punto de operación. El piso de recall de 0.70 se cumplió en validación (0.811) y en test (0.7632). No se reajustó el umbral con test para no invalidar la evaluación.

Matriz de confusión (test, umbral 0.51). TN = 508, FP = 221, FN = 9, TP = 29. El modelo detecta 29 de 38 casos con ACV y pierde 9 (23.7%). Con 0.50 detecta 30 y pierde 8 (21.1%), pero emite 228 falsas alarmas en lugar de 221.

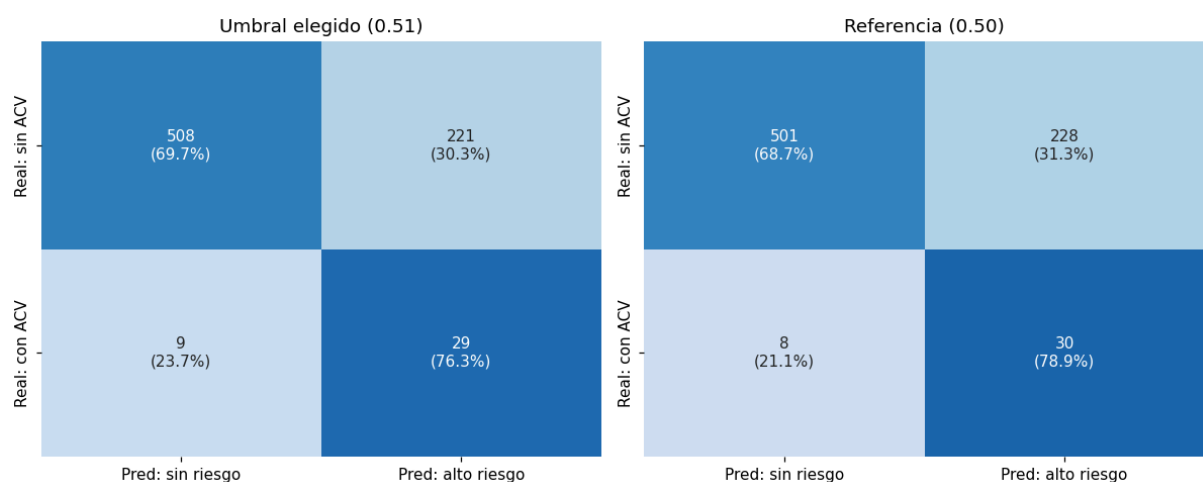


Figura 3. Matrices de confusión en test.

Impacto de los falsos negativos en un contexto clínico. Un falso negativo es un paciente con alto riesgo que recibe "sin riesgo". El modelo no dispara ningún control ni medida preventiva. El paciente y el médico reciben una tranquilidad falsa. Un falso positivo cuesta una consulta o un estudio extra. Por esta asimetría, los falsos negativos pesan más. Es la razón del piso de recall al elegir el umbral.

Qué aporta el modelo. Una alerta eleva la probabilidad de ACV de 5.0% (tasa base en test) a 11.6%. Pero cada caso real cuesta unas 9 alertas. Un "sin riesgo" baja la probabilidad a 1.7%, sin llegar a cero. El modelo sirve

para priorizar controles y no para descartar un ACV.

Ablación: qué decisiones importaron

Se reentrenó el modelo con una decisión cambiada a la vez, con 5 y luego 10 semillas, solo en validación. Las tablas completas están en el notebook, sección "Ablación". Sin `pos_weight`, el umbral 0.5 no marca a ningún paciente, pero la precisión con el umbral elegido no cambia. Leaky ReLU no mostró ventaja frente a ReLU, ni la inicialización Kaiming frente al default de PyTorch. Cinco tamaños de red, de 449 a 46081 parámetros, y Dropout de 0.0 a 0.5 dan un AUC de validación equivalente. Solo el indicador de faltante de `bmi` superó dos errores estándar (+0.0062), una señal débil. La regresión logística (0.8425) supera a la base del MLP (0.8337), sin ser concluyente con 37 positivos.

Desafíos encontrados

- **Pocos positivos.** Con 249 casos en total, las métricas de validación y test son ruidosas. Un paciente mueve el recall unos 2.7 puntos. El recall de validación es unos 5 puntos mayor que el de test.
- **El umbral 0.5 no es neutral.** `pos_weight` empuja las probabilidades hacia arriba. Con 0.5, el modelo marca a 258 de 767 pacientes de test. Fue necesario elegir un umbral de forma explícita y justificarlo.
- **Dos hipótesis propias no se sostuvieron.** Se pensó que `pos_weight` aumentaba el riesgo de neuronas muertas, y que una red más grande sobreajustaba más. Al medirlo, no hubo diferencia con ReLU, ni entre tamaños de red. Se corrigió el texto del notebook y de este informe.
- **El criterio de parada es frágil.** En la variante base, la época elegida va de 2 a 6 según la semilla, y el modelo principal eligió la 10.
- **Confusión con la edad.** Varias variables categóricas son en parte proxies de la edad. Esto limita la interpretación del modelo.

Limitaciones

- Solo 174, 37 y 38 positivos en train, validación y test. El resultado sale de una división y una semilla, sin intervalo de confianza.
- Las salidas no son probabilidades de riesgo, porque `pos_weight` las distorsiona.
- Las ablaciones descartan efectos de más de unos 0.007 de AUC, y no efectos chicos.
- No se midieron la tasa de aprendizaje, el tamaño del batch, la regularización L2 ni otra división de los datos.

Conclusiones

- Para un problema binario con desbalance fuerte, la accuracy no sirve: una regla que siempre predice "sin ACV" tiene 0.9505. Se usó AUC-ROC para comparar modelos, y recall y precisión para el punto de operación.
 - La regla del umbral eligió 0.51, casi igual a 0.50. Pasar de 0.50 a 0.51 baja las falsas alarmas de 228 a 221 y el recall de 0.7895 a 0.7632. Elegir el umbral es una decisión clínica.
 - Separar validación y test dio una estimación honesta: el recall cayó unos 5 puntos de una a otro.
 - La medición cambió las conclusiones. Cinco decisiones que se justificaron con argumentos no mostraron un efecto medible en el AUC de validación. Son `pos_weight`, Leaky ReLU, la inicialización, el tamaño de la red y el Dropout. Conviene medir antes de defender una decisión de diseño.
 - El MLP aprende señal real (AUC-ROC 0.8177 en test). Pero no supera a un modelo lineal en validación. En estos datos, el límite parece estar en la cantidad de casos positivos y en las variables, no en la arquitectura.
 - El modelo sirve para priorizar controles, no para descartar un ACV: un "sin riesgo" baja la probabilidad de 5.0% a 1.7%, sin llegar a cero.
-

Problema 2: Clasificación Multiclase (dígitos MNIST)

Procedimiento

El problema se resolvió en `problema2.ipynb`, en este orden:

1. Carga y verificación de los archivos IDX.
2. Análisis exploratorio: muestra de imágenes, distribución de clases e histograma de píxeles.
3. Preprocesamiento: división en train, validación y test, aplanamiento, normalización y codificación de las etiquetas.
4. Diseño del MLP en PyTorch (784 -> 256 -> 128 -> 10).
5. Entrenamiento con Adam y Early Stopping, y curvas de aprendizaje.
6. Evaluación en test: accuracy global y por clase.
7. Matriz de confusión y análisis de errores.
8. Experimento de desplazamiento, ablación de decisiones y reflexión sobre las limitaciones del MLP.

Decisiones de diseño y justificación

Carga y preprocesamiento

Los archivos IDX se leen con el encabezado en formato *big-endian*, como indica la guía del enunciado. Un `assert` verifica el número mágico de cada archivo (2051 para imágenes y 2049 para etiquetas). Otros `assert` verifican las dimensiones y el tipo de dato. Train tiene 60000 imágenes y test tiene 10000, todas de 28 x 28 píxeles en `uint8`.

Las clases están casi balanceadas, con una razón de 1.24 entre la clase más grande y la más chica. En el Problema 1 la razón era 19.5. Por eso no se aplicó ninguna corrección de desbalance.

Se separaron 6000 imágenes de train para validación, con estratificación por dígito. Las 10000 imágenes oficiales de test se usan una sola vez.

Píxeles. Cada imagen se aplanó a 784 valores. Los valores se dividieron por 255 para llevarlos a [0, 1], como indica la guía. El apunte de la Unidad 5 explica la razón. Los píxeles en [0, 255] producen gradientes mucho mayores que los valores en [0, 1]. Eso dificulta el entrenamiento. Dividir por 255 usa una constante fija, así que no hay fuga de información entre conjuntos.

Etiquetas: enteros en lugar de one-hot. Se usan etiquetas enteras (0 a 9) con `nn.CrossEntropyLoss`. Con una etiqueta one-hot, la cross-entropy categórica se reduce a $-\log p$ de la clase verdadera, porque solo un término vale 1. El índice entero selecciona ese mismo término, y la pérdida y los gradientes son idénticos. Se comprobó de forma numérica: sobre los mismos logits aleatorios, las dos formas dan 2.71831250, con diferencia 0. El apunte justifica one-hot para no imponer un orden entre las clases. Esa razón vale cuando el entero es una entrada o un objetivo numérico. Aquí el entero solo indica qué salida de la red entra en la pérdida.

Arquitectura del modelo

MLP de 2 capas ocultas, 784 -> 256 -> 128 -> 10, con 235146 parámetros. Se usan las 256 y 128 unidades del ejemplo de la Unidad 4. Es un tamaño moderado y no fue optimizado.

- **Activación oculta: ReLU** con inicialización Kaiming. ReLU es el valor por defecto del apunte de la Unidad 1 para capas ocultas, y Kaiming es la inicialización que recomienda la Unidad 3.
- **Salida: 10 logits, sin activación.** Softmax es la activación de salida para clases excluyentes. `CrossEntropyLoss` la aplica internamente y espera logits crudos. Para predecir se usa el `argmax` de los logits.
- **Optimizador:** Adam con tasa de aprendizaje 1e-3, el valor por defecto del apunte de la Unidad 3. El batch es de 128 imágenes.

- **Regularización:** Dropout de 0.3 después de cada capa oculta, y Early Stopping sobre la pérdida de validación (paciencia 10). El apunte de la Unidad 4 indica que, con datasets grandes, bastan tasas de 0.3 o menos.

Entrenamiento

El modelo se entrenó con un máximo de 50 épocas. El entrenamiento paró en la época 21. El modelo conserva los pesos de la época 11, con `val_loss` 0.0776 y `val_acc` 0.9797.

Las curvas muestran sobreajuste moderado y ningún subajuste. La pérdida de validación llega a su mínimo en la época 11 y después sube. En la mejor época, la brecha entre train y validación es de 1.54 puntos.

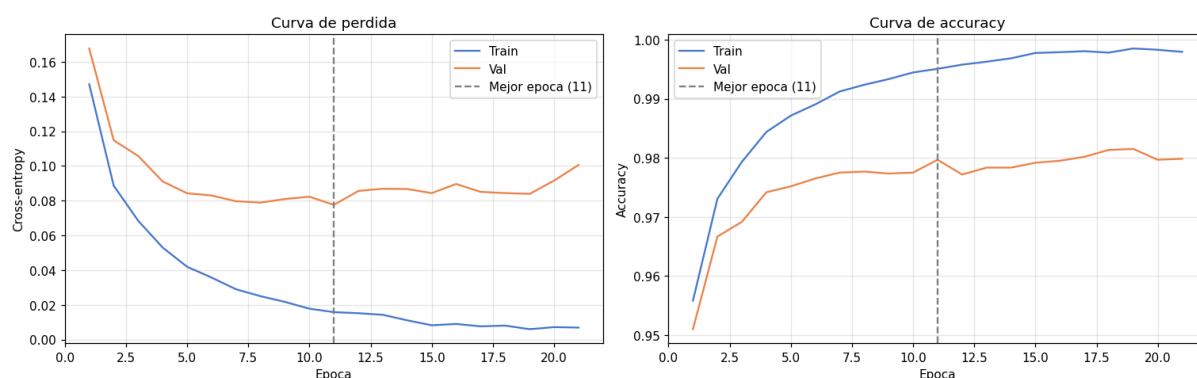


Figura 4. Curvas de aprendizaje de train y validación.

Evaluación en test

El modelo acierta 9802 de 10000 imágenes de test. La accuracy es 0.9802, con un intervalo de confianza de 95% de [0.9773, 0.9828]. La accuracy de validación (0.9797) cae dentro de ese intervalo. La accuracy por clase va de 0.9682 (dígito 8) a 0.9929 (dígito 0). Los dígitos más difíciles son el 8, el 5 (0.9709) y el 9 (0.9752). La prueba chi-cuadrado rechaza que las 10 accuracies sean iguales ($p = 0.0004$).

La accuracy es adecuada como métrica global, con tres cautelas. Las clases están balanceadas, así que no se infla por ignorar una clase. Oculta diferencias reales entre clases, de hasta 2.5 puntos. Tampoco dice nada sobre la confianza de las predicciones.

Análisis de errores

Al sumar las dos direcciones, el par más confundido es 4-9 (16 imágenes). Siguen 2-7 (14), 3-5 y 7-9 (12 cada uno), 7-8 (10) y 3-9 (9). Con tan pocas imágenes por par, el orden entre los pares no es fiable. Ningún par incluye al 0 ni al 1, que tienen la mayor accuracy.

Se miraron las 24 imágenes de las cuatro confusiones más frecuentes. Lectura de las imágenes (no probada):

- **4 -> 9:** los trazos de arriba del 4 se juntan y forman un lazo cerrado, como el de un 9.
- **5 -> 3:** el trazo vertical de arriba del 5 es corto o casi no se ve. La barra de arriba y la panza forman un 3.
- **7 -> 2:** el 7 tiene una barra que cruza el trazo, o un trazo al pie, que se parece a la base de un 2.
- **9 -> 7:** el lazo de arriba del 9 está abierto o es muy chico, como en un 7.

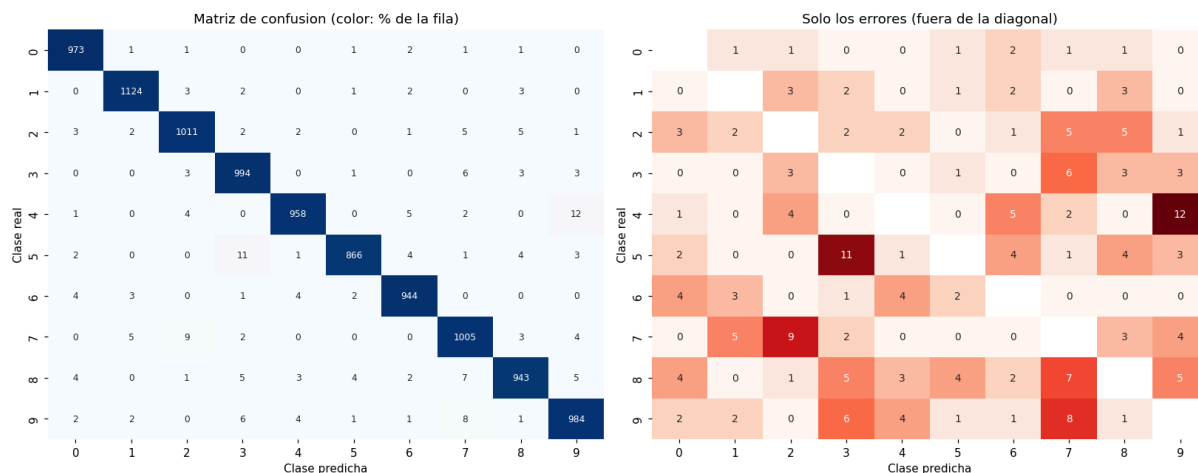


Figura 5. Matriz de confusión en test.

La confianza es informativa, pero no es confiable. Es 0.9928 en promedio en los aciertos y 0.7597 en los errores. Aun así, 70 de los 198 errores (35%) tienen confianza mayor que 0.9. Softmax reparte siempre una probabilidad total de 1, y la red no tiene una salida "no sé". Un umbral de confianza de 0.9 marca el 3.3% de test y contiene el 65% de los errores.



Figura 6. Errores con alta confianza.

Experimento de desplazamiento

Apunte de la Unidad 5, sección 5.1.2

En una red densa, el conocimiento espacial no se transfiere: un patrón aprendido en una región no sirve para reconocerlo en otra posición.

— Unidad 5 - Redes Convolucionales - Parte 1, sección 5.1.2

Se midió esa afirmación. La predicción se escribió antes de medir: la accuracy baja cuando se desplazan los dígitos, porque el entrenamiento solo vio dígitos centrados. Las imágenes de test se desplazaron de 1 a 4 píxeles, en cuatro direcciones y con relleno de ceros. Se evaluó el mismo modelo, sin reentrenar.

Desplazamiento	Accuracy (promedio de 4 direcciones)	Caída
0 px	0.9802	-
1 px	0.9642	1.6 puntos
2 px	0.8540	12.6 puntos
3 px	0.5953	38.5 puntos
4 px	0.3354	64.5 puntos

La predicción se cumple. Un desplazamiento de 2 píxeles, menos del 8% del ancho de la imagen, ya cuesta 12.6 puntos. Un control repitió la medición solo sobre las imágenes cuyo trazo sobrevive intacto, y la accuracy cae casi igual. La caída se debe a la posición, no al recorte (ver el notebook, sección 8).

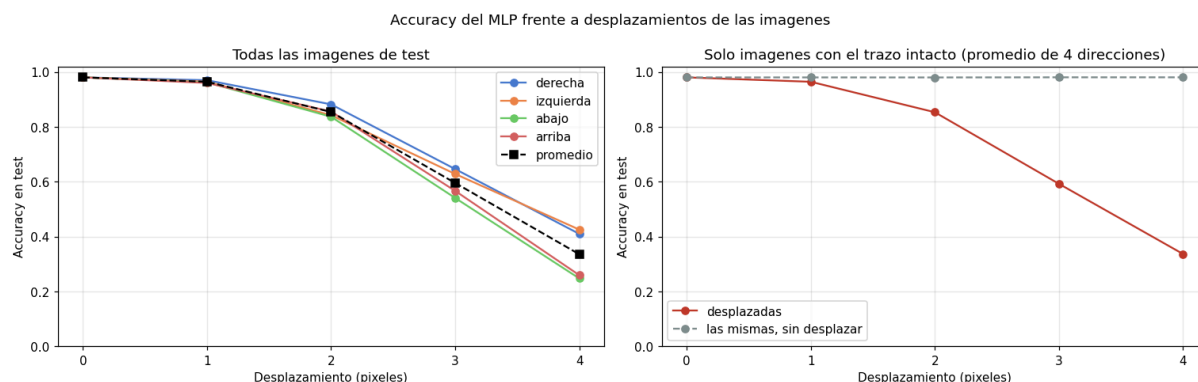


Figura 7. Accuracy en test al desplazar las imágenes.

Ablación: qué decisiones importaron

Se midieron el tamaño de la red, el Dropout y el criterio de parada. Se cambió una decisión a la vez, con 5 semillas por variante. El tamaño importa solo hacia abajo: la red de 64-32 pierde 1.05 puntos, y la de 512-256 casi no gana. Quitar el Dropout cuesta 0.33 puntos, un efecto chico pero consistente en las 5 semillas. Elegir la época por accuracy gana por poco, con una prueba optimista. Las diferencias menores que unos 0.002 no se distinguen del ruido. La tabla completa está en el notebook.

Reflexión: limitaciones del MLP frente a imágenes

1. **No tolera cambios de posición (medido).** Al aplanar, el vector no distingue un píxel de su vecino, como explica el apunte de la Unidad 5.
2. **Los parámetros crecen con el tamaño de la imagen.** La primera capa de este modelo tiene el 85% de los pesos. Un detector aprendido en una posición no se reutiliza en otra.
3. **No compone rasgos locales (no probado).** Las confusiones son de rasgos locales, como un lazo abierto.
4. **Más ancho casi no ayuda (medido).** La red de 512-256 no muestra una diferencia medible en la mitad B.

Las redes convolucionales aportan localidad, compartición de pesos y jerarquía (Unidad 5, sección 5.1.4). El mismo detector recorre todas las posiciones, y eso da equivarianza a la traslación. Por eso se espera que una CNN pierda mucho menos que 12.6 puntos con 2 píxeles. Es una hipótesis, y no se midió en este trabajo.

Desafíos encontrados

- **La pérdida y la accuracy de validación no eligen la misma época.** La pérdida elige la 11 y la accuracy llega a su máximo en la 19. Se mantuvo la regla fijada antes de entrenar.
- **Reproducibilidad.** La semilla 42 daba otra época en la ablación que en el entrenamiento principal. Cada recorrido de un `DataLoader` sin generador propio consumía números del generador global, que también usa el Dropout. La solución da a cada `DataLoader` su propio generador. Una ejecución en CPU no se

repitió, y la causa no se encontró. Las cifras de este informe son de una ejecución en GPU en la que el chequeo pasa.

- **Una suposición propia era falsa.** Se supuso que desplazar una imagen 1 píxel y volver no perdía nada. Algunas imágenes de test tienen trazo en el borde. Por eso se agregó el control sobre imágenes con el trazo intacto.

Conclusiones

- Un MLP chico (235146 parámetros) llega a 0.9802 de accuracy en test, con un intervalo de confianza de 95% de [0.9773, 0.9828]. Las clases están balanceadas, así que la accuracy es una métrica adecuada. Las diferencias por clase, de 0.9682 (dígito 8) a 0.9929 (dígito 0), son reales.
 - Los errores se concentran en dígitos de forma parecida: 4-9, 2-7, 3-5, 7-9, 7-8 y 3-9.
 - La red es frágil frente a la posición. Un desplazamiento de 2 píxeles baja la accuracy 12.6 puntos. Esto respalda la afirmación del apunte de la Unidad 5 sobre aplanar la imagen.
 - De las decisiones de diseño, el tamaño mostró el efecto más claro, y solo hacia abajo. Quitar el Dropout tiene un efecto chico pero consistente.
 - La confianza es informativa, pero no es confiable. No sirve para certificar una predicción.
-

Problema 3: Regresión (Profit por transacción, Superstore)

Procedimiento

El problema se resuelve en `problema3.ipynb`, en este orden:

1. Carga y análisis exploratorio: distribución de `Profit`, extremos, predictores y cardinalidad.
2. Preprocesamiento: columnas, fechas, división por pedido, codificación y escalado.
3. Diseño del MLP y comparación de las funciones de costo MSE, MAE y Huber.
4. Entrenamiento, corrección del sobreajuste y ablación de decisiones.
5. Evaluación en dólares con bootstrap, y análisis de residuos.
6. Comparación con Ridge y con gradient boosting, costo y conclusiones.

Decisiones de diseño y justificación

Análisis exploratorio

El CSV se lee con `latin-1`. Quedan 9994 filas, sin faltantes ni duplicados.

Profit tiene colas extremadamente pesadas. La mediana es 8.67 y la media 28.66. El desvío es 234.26, unas 8.5 veces el rango intercuartil. El rango va de -6599.98 a 8399.98. La curtosis en exceso es 397.19. El 18.7% de las filas tiene `Profit` negativo.

El 1% de las filas con mayor `|Profit|` concentra el 28.1% del `|Profit|` total. Los extremos son eventos reales: copiadoras sin descuento, o `Machines` y `Binders` con descuentos altos. Se mantienen todas las filas. Los extremos se tratan con una función de costo y un escalado robustos.

`Profit` crece casi en proporción a `Sales`, y `Discount` fija el signo. Con descuento 0, ninguna fila pierde. Con 0.30 pierde el 91.63%. La correlación lineal subestima a `Discount`, porque el `Profit` cambia de signo como un escalón.

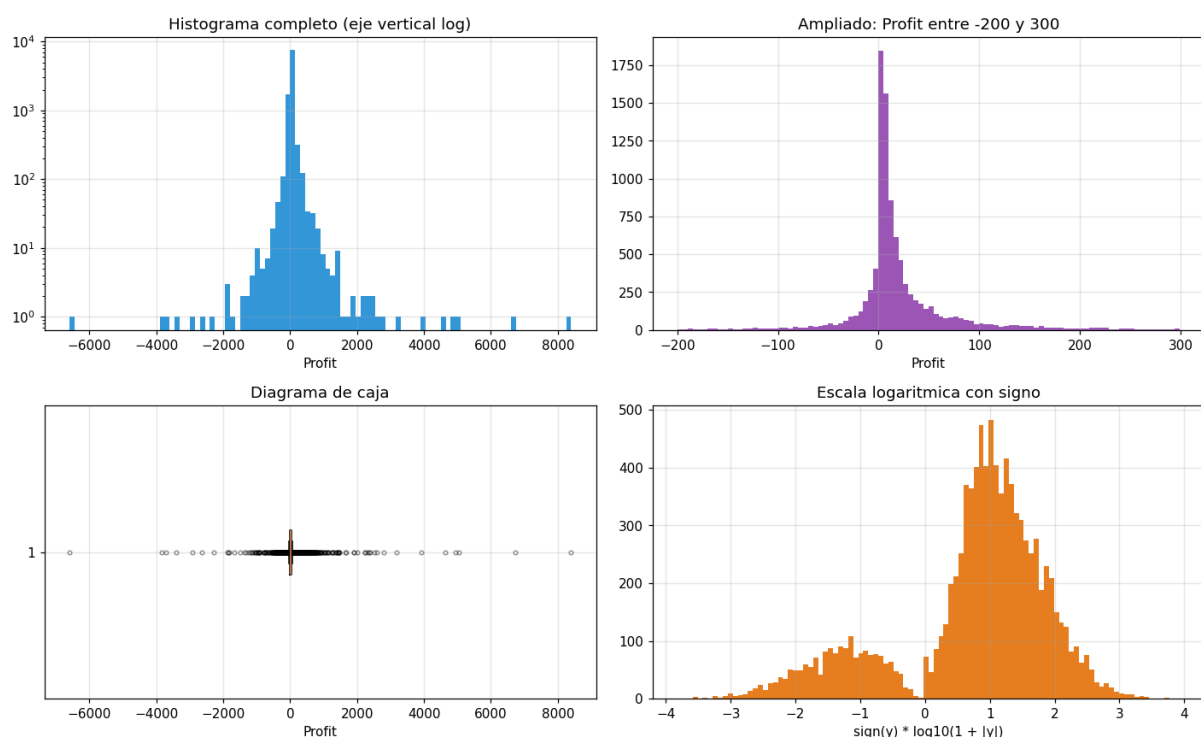


Figura 8. Distribución de Profit.

Preprocesamiento

Columnas eliminadas (9). Se eliminan los identificadores, `Country` (constante), y `City` y `Postal Code` (cientos de niveles con pocas filas). `State` y `Region` conservan la geografía.

Fechas. De `Order Date` se extraen año, mes, día, trimestre y día de la semana. Se agrega la demora de envío en días, que va de 0 a 7.

División por pedido. Los 9994 registros vienen de 5009 pedidos. Las filas de un pedido comparten cliente, fechas, modo de envío y estado. Con una división aleatoria por filas, el 64.9% de las filas de test tiene su pedido también en train. Por eso se dividió por `Order ID` con `GroupShuffleSplit`: 70%, 15% y 15% de los pedidos. Quedan 7051 filas en train, 1431 en validación y 1512 en test, con fuga de 0.0%. Los clientes y productos sí se repiten, y es legítimo: los pedidos futuros vienen de clientes y productos conocidos.

De las 20 filas con mayor `|Profit|`, solo 4 cayeron en test. Por eso RMSE y R2 en test dependen de pocas filas, y se usan intervalos con *bootstrap*.

Codificación por cardinalidad. One-hot para `Ship Mode`, `Segment`, `State`, `Region`, `Category` y `Sub-Category`. Con las 9 numéricas dan 88 entradas. Se descartó *target encoding*: con colas tan largas, la media de un nivel chico está dominada por una o dos filas.

Escalado. El escalador se ajusta solo con train. `Sales` tiene asimetría 14.19. `RobustScaler` da al centro 3.4 veces más espacio que `StandardScaler`, y se eligió como punto de partida. La ablación posterior lo confirmó: `StandardScaler` empeora el MAE de validación en 1.53 (ver el notebook, sección de ablación).

Diseño de la red

- **Salida lineal**, una neurona. El apunte de la Unidad 2 (sección 2.4.1) indica que en regresión la salida no lleva Sigmoid, Softmax ni ReLU. `Profit` es negativo en el 18.7% de las filas.
- **Objetivo escalado** con la media y el desvío de train (27.93 y 243.40). Todas las métricas se calculan en dólares.
- **Arquitectura** 88 -> 64 -> 32 -> 1, con ReLU y 7809 parámetros. Adam con tasa `1e-3`, y parada por el MAE de validación.
- **Reproducibilidad.** Cada corrida fija su semilla, baraja con su propio generador y evalúa por tensores. Una prueba verifica que dos corridas dan predicciones idénticas.

Apunte de la Unidad 2, conclusiones clave del tema 2.5

El desbalance de escala entre pérdidas de distinta naturaleza (CCE vs. MSE) es el principal riesgo práctico: siempre normalizar los targets de regresión y ajustar los λ k cuidadosamente.

— Unidad 2 - Redes Feed Forward - Parte II, tema 2.5

El apunte da este consejo para redes con varias salidas. Aquí se aplica por otra razón. Al inicio, la red da salidas de escala 1, como el objetivo en z-score. En dólares, la red tiene que llegar a cientos o miles.

Referencia en validación: predecir la mediana de train da MAE 57.11.

Función de costo: MSE, MAE y Huber

Se entrenó la misma red con cada función de costo, con 10 semillas, sobre validación. Antes de medir se fijó la regla de decisión: la función con menor MAE medio.

Función de costo	MAE	RMSE	MedAE	R2	MAE con abs(Profit) > 100
MSE	20.79 ± 0.85	52.11 ± 2.43	8.33 ± 0.86	0.927	78.38
MAE	18.23 ± 0.47	56.28 ± 2.71	3.98 ± 0.41	0.915	80.23
Huber delta 0.1	19.04 ± 0.53	55.49 ± 3.48	5.65 ± 0.44	0.917	79.30
Huber delta 1.0	19.70 ± 0.76	51.89 ± 2.62	7.12 ± 0.97	0.928	76.61

Se eligió MAE. Mejora a MSE en 2.55 (error estándar 0.34) y gana en las 10 semillas. A cambio, empeora el RMSE en un 8%. La ganancia está en las transacciones típicas. En las grandes, el error medio es de 77 a 80 dólares con todas las funciones.

Corrección del sobreajuste

En la mejor época, el MAE de train es 12.72 y el de validación 18.38. Hay sobreajuste moderado y no hay subajuste. Se probaron Dropout y L2 de la Unidad 4, con 10 semillas. Se fijó una regla: adoptar solo una mejora mayor que 2 errores estándar pareados.

Se adoptó L2 con `weight_decay = 1e-3`. El MAE de validación baja de 18.23 a 16.39, en las 10 semillas. Dropout no ayuda. La grilla de L2 se amplió al ver el óptimo en el borde, y la selección es algo optimista. Con la semilla 0, la brecha entre train y validación baja de 5.66 a 3.08.

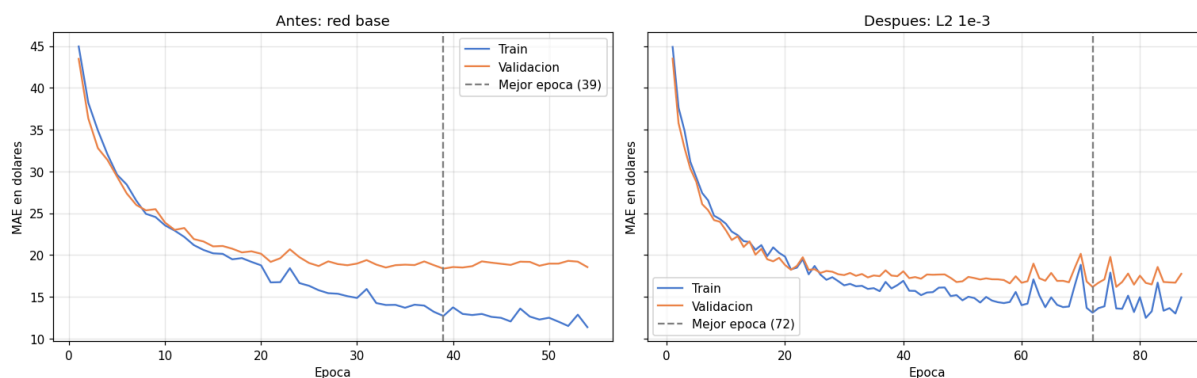


Figura 9. Curvas antes y después de la corrección del sobreajuste.

La ablación con la configuración final no cambió ninguna decisión. Quitar `State` o cambiar el tamaño de la red no supera la regla (ver el notebook, sección de ablación).

Evaluación en test

Se evaluó el MLP final con la semilla 0, fijada antes de ver test. Los intervalos de 95% salen de 10000 remuestreos *bootstrap* de los pedidos de test.

Métrica	Observado	IC95 por pedidos
MAE	17.08	[14.00, 20.86]
RMSE	70.48	[38.18, 97.69]
MedAE	2.95	[2.64, 3.31]
R2	0.903	[0.804, 0.965]
Sesgo (predicho - real)	+3.51	[+0.42, +7.23]

La mediana de train da un MAE de 59.94. El MLP lo reduce en un 72%. El MAE es estable y el RMSE no. Las 10 semillas dan un MAE medio de 17.68.

Lectura para el negocio. La mitad de las transacciones tiene un error menor o igual que 2.9 dólares, y el 72.8% menor que 10. El modelo sobreestima el `Profit` total de test en un 10.9%. De las 275 transacciones con pérdida, detecta el 84.0%, con una precisión de 86.2%.

Análisis de residuos

El error crece con la magnitud del `Profit`. Las 201 filas con `|Profit|` de 100 o más concentran el 58.0% del error absoluto. El modelo subestima las pérdidas grandes en un 30%. Las 10 peores filas aportan el 83.4% del error cuadrático, y el RMSE es casi el error de esas 10 transacciones. En los 5 peores casos, el residuo es un error de margen multiplicado por `Sales`. Son sobre todo combinaciones raras de subcategoría y descuento, como `Machines` con descuento de 0.7.

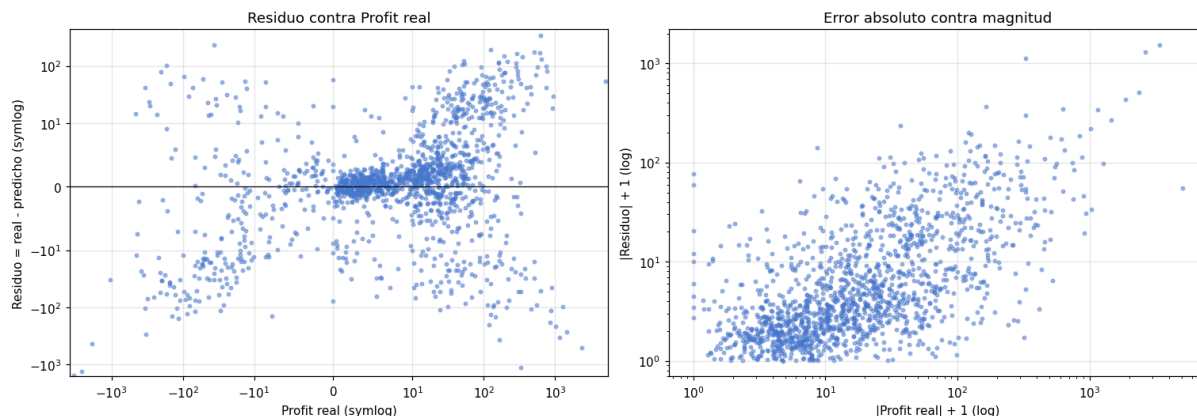


Figura 10. Error según la magnitud del `Profit`.

Comparación con otros modelos y costo

Apunte de la Unidad 0, conclusiones clave (punto 5)

Siempre establecer una baseline simple (regresión logística, XGBoost) antes de recurrir al DL. Si la baseline es competitiva, el overhead del DL raramente se justifica.

— Apunte - Unidad 0 - Introducción (apuntes/)

El apunte pide una línea base simple antes de un modelo de DL. El MLP se compara con Ridge, con regresión lineal L1, y con `HistGradientBoostingRegressor` con error absoluto y con error cuadrático. No se usaron XGBoost ni LightGBM porque no están instalados en el entorno.

Cada modelo ajusta sus hiperparámetros sobre validación. Se ampliaron dos grillas del boosting porque el mejor valor estaba en el borde. Esta ampliación ocurrió después de ver un primer resultado en test, y el ajuste es algo optimista.

Modelo	MAE	RMSE	MedAE	R2	Dif. MAE vs MLP	Dif. RMSE vs MLP
Ridge	47.21	188.05	12.63	0.31	+30.11 [+23.66, +37.88]	+115.16 [+58.50, +170.62]
Lineal L1	41.84	182.75	8.08	0.35	+24.73 [+18.66, +31.85]	+109.79 [+54.98, +160.79]
Boosting, error abs.	21.37	120.59	1.91	0.72	+4.28 [+0.65, +9.57]	+47.61 [+3.79, +103.65]
Boosting, error cuad.	24.78	139.58	2.42	0.62	+7.68 [+3.33, +13.85]	+66.56 [+19.00, +126.91]
MLP (semilla 0)	17.08	70.48	2.95	0.90	-	-

- **Modelos lineales.** Quedan muy atrás. `Profit` es un margen multiplicado por `Sales`, y un modelo lineal no representa un producto.
- **MLP frente a boosting con error absoluto.** El MLP es mejor en MAE por 4.28 dólares, con un límite bajo cercano a 0. Con la grilla chica, el intervalo era $[-0.08, +8.38]$ y contenía el 0.
- **Por tamaño.** En las transacciones grandes gana el MLP (74.46 frente a 113.14). En las típicas gana el boosting con error absoluto (7.31 frente a 8.28).

Costo. El MLP entrena en 24.5 s y predice unas 250 veces más rápido que el boosting. El costo está en el diseño: 340 entrenamientos (unos 139 minutos), frente a 728 segundos para los cuatro modelos de comparación.

¿Se justifica el MLP? Frente a los modelos lineales, sí. Para la transacción típica, el boosting es competitivo y algo mejor. Según el apunte, en ese caso el costo del DL rara vez se justifica. Se prefiere el MLP porque los errores en transacciones grandes cuestan más dólares. Es un juicio sobre el costo de los errores, que los datos no verifican.

Limitaciones

- El error se concentra en pocas transacciones, y el RMSE y el R2 de test tienen intervalos muy anchos.
- La división es por pedido y no por fecha. No se midió una división temporal.
- Las grillas se ampliaron mirando resultados, y eso agrega optimismo.

Desafíos encontrados

- **Los extremos no se reparten por igual.** Solo 4 de las 20 filas más extremas cayeron en test. Por eso se eligió el MAE como métrica principal y se usó *bootstrap*.
- **La fuga por pedido.** La división aleatoria por filas deja el 64.9% de las filas de test en pedidos de train. Se dividió por pedido.
- **Óptimos en el borde de la grilla.** Pasó con L2 y con dos grillas del boosting. Las del boosting se ampliaron después de ver el primer resultado de test. Así, la ventaja del MLP en MAE pasó de no significativa a significativa por poco.
- **Una predicción falló.** Se esperaba que el bootstrap por filas diera intervalos más angostos que por pedidos. Son iguales.
- **Una ablación que medía otra cosa.** La ablación de escaladores cambia las 9 variables numéricas, y no solo `Sales`. Se agregó una comparación que aísla `log1p`, y se corrigió la conclusión.

Conclusiones

- Un MLP de 7809 parámetros, con función de costo MAE y L2, llega a un MAE de test de 17.08 [14.00, 20.86]. Es un 72% menos que el de la mediana de train.
 - La función de costo MAE mejora el error típico frente a MSE, a cambio de un RMSE peor. La regularización L2 mejoró el MAE en 1.84.
 - El error crece con la magnitud del `Profit`. El modelo subestima en un 30% las pérdidas grandes.
 - Frente a los modelos lineales, el MLP es claramente mejor. Frente al boosting, es mejor en las transacciones grandes y competitivo en la típica.
 - El costo del MLP está en el diseño y no en la ejecución.
-

Problema 4: Red Neuronal Convolutacional

Procedimiento

1. Selección de 8 razas del dataset *70 Dog Breeds Image Dataset*.
2. Carga a 128 x 128, control de repetidas, normalización y data augmentation.
3. Cinco arquitecturas (E1 a E3) y tres variantes de regularización (E4), con 3 semillas cada una.
4. Diagnóstico de la falla del Dropout, y E5: cuatro variantes que la corrigen.
5. Elección del mejor modelo por validación, y análisis en test.

El notebook es `tp1/problema4.ipynb`. Se ejecuta en una GPU de Google Colab.

Decisiones de diseño y justificación

Datos y preprocesamiento

El enunciado recomienda de 5 a 10 razas. Se eligieron 8, en tres grupos:

Grupo	Razas	Por qué
Trío similar	Border Collie, Collie, Bearded Collie	Pelo largo y colores parecidos. Es la parte difícil.
Par similar	Labrador, Golden Retriever	Misma talla y cabeza parecida.
Distintas	Pug, Siberian Husky, Chihuahua	Cara, talla y pelaje muy diferentes. Es la parte fácil.

- **Validación y test son chicos:** 80 imágenes por conjunto. Un acierto cambia la exactitud en 1.25 puntos. Por eso se usan 3 semillas.
- **Imágenes repetidas.** El hash MD5 mostró 8 imágenes de train repetidas, 5 de ellas en test. Se quitaron de train. Train queda con 920 imágenes.
- **Resolución de 128 x 128.** Con 224 x 224 el costo es unas 3 veces mayor.
- **Normalización por canal** con la media y el desvío de train solamente.
- **Data augmentation solo en train:** volteo horizontal, rotación de hasta 15 grados, zoom, traslación, contraste y brillo. No hay volteo vertical, porque un perro boca abajo no aparece en datos reales.



Figura 11. Una imagen original de train (izquierda) y siete transformaciones.

Arquitecturas y entrenamiento

Todas las redes usan bloques Conv-BN-ReLU, una cabeza densa de 128 neuronas, salida lineal de 8 logits, `CrossEntropyLoss` e inicialización Kaiming.

Experimento	Estructura	Cambio aislado
E1	3 bloques Conv3x3-BN-ReLU-MaxPool (32, 64, 128)	Línea base
E2a	4 etapas de 2 convoluciones 3x3 (32 a 256), con MaxPool	Profundidad
E2b	E2a con filtros de 5x5	Tamaño del filtro
E2c	E2a con stride 2 en lugar de MaxPool	Stride
E3	E2a con una conexión residual en cada etapa	Bloques residuales
E4-D, E4-A, E4-DA	La mejor de E1 a E3, con Dropout, con augmentation, o con los dos	Regularización
E5a, E5b, E5c	E3 con cabeza de <i>global average pooling</i> , con augmentation	Cabeza, Dropout y entrenamiento largo
E5d	E4-DA con Leaky ReLU en la cabeza	Activación de la cabeza

Cada experimento cambia una sola cosa, para atribuir cada diferencia a ese cambio. Se eligieron bloques residuales y no Inception, porque son más simples.

- **Entrenamiento:** Adam con tasa $1e-3$, lotes de 32 y parada temprana con paciencia 10 y un máximo de 50 épocas.
- **Reglas fijadas antes de medir.** La elección usa la validación promedio de 3 semillas. Se escribieron predicciones (P1 a P10) antes de entrenar.
- **Checkpoints cada 5 épocas**, con el estado de los generadores aleatorios. Una prueba verifica que una corrida retomada da el mismo historial.

Resultados

Cada fila es el promedio de 3 semillas, con el desvío. La época es la elegida por la parada temprana.

Experimento	Acc. train	Acc. val	Acc. test	# Parámetros	Épocas	Observaciones
E1	0.742 ± 0.131	0.508 ± 0.095	0.500 ± 0.045	4288936	10	Brecha train-val 0.23. Val de 0.40 a 0.57.
E2a	0.657 ± 0.064	0.504 ± 0.072	0.496 ± 0.064	3271528	10	La profundidad no mejora a E1.
E2b	0.615 ± 0.209	0.488 ± 0.109	0.471 ± 0.097	5353832	23	Filtros de 5x5. 150 s por corrida, la más inestable.
E2c	0.799 ± 0.174	0.475 ± 0.025	0.550 ± 0.066	3271528	7	Stride 2. 14 s por corrida, la más rápida.
E3	0.912 ± 0.086	0.525 ± 0.062	0.633 ± 0.026	3315592	9	Residual. Más ajuste a train, brecha 0.39. Base de E4.
E4-D	0.366 ± 0.048	0.279 ± 0.040	0.225 ± 0.033	3315592	25	Dropout 0.5. La red no aprende train.
E4-A	0.881 ± 0.020	0.683 ± 0.029	0.671 ± 0.040	3315592	40	Augmentation. Brecha 0.20. Una corrida en el tope de 50.
E4-DA	0.175 ± 0.093	0.175 ± 0.076	0.171 ± 0.051	3315592	3	Dropout y augmentation. Cerca del azar (0.125).
E5a	0.793 ± 0.098	0.700 ± 0.119	0.679 ± 0.047	1219336	27	Cabeza GAP. Igual a E4-A dentro del ruido.
E5b	0.867 ± 0.049	0.767 ± 0.036	0.742 ± 0.014	1219336	35	GAP y Dropout 0.3. El Dropout ya no colapsa.
E5c	0.995 ± 0.002	0.871 ± 0.026	0.825 ± 0.012	1219336	71	GAP, 100 épocas, coseno. El mejor. 286 s por corrida.
E5d	0.682 ± 0.019	0.625 ± 0.050	0.617 ± 0.040	3315592	47	E4-DA con Leaky ReLU. Aprende. Tres corridas en el tope de 50.

- **Arquitectura (E1 a E3).** Las cinco arquitecturas quedan entre 0.475 y 0.525 de validación, con diferencias de 1 error estándar o menos.
- **Bloques residuales.** E3 ajusta más a train que E2a, pero no mejora la validación.
- **Augmentation (E4-A).** Mejora a E3 por 0.158 en validación, 3.1 errores estándar, en las 3 semillas. La brecha con train baja de 0.39 a 0.20.
- **Dropout (E4-D y E4-DA).** Rompe el entrenamiento: la red no aprende ni train.

Diagnóstico de la falla del Dropout. Se midió después de ver los resultados. Se entrenó E3 durante 10 épocas y se contaron las neuronas muertas de la cabeza densa (128 ReLU). Una neurona muerta da 0 para todas las imágenes y ya no recibe gradiente. La mayor parte muere en la primera época, y ninguna vuelve. Sin Dropout sobreviven 10 a 16 neuronas, y alcanzan para aprender. Con Dropout y augmentation mueren las 128, y la red da la misma salida para todo. Una tasa 10 veces menor no evita las muertes (ver el notebook, sección del diagnóstico).

E5: una corrección guiada por el diagnóstico. E5a reemplaza la cabeza densa por *global average pooling* (GAP) y una capa lineal. La cabeza baja de 2.1 millones de parámetros a 2056, sin neuronas ReLU ocultas. E5b agrega Dropout 0.3. E5c entrena 100 épocas con paciencia 20 y decaimiento coseno. E5d es E4-DA con Leaky ReLU.

La cabeza GAP sola (E5a) no mejora a E4-A, pero usa un 63% menos de parámetros. Con ella, el Dropout ya no colapsa (E5b). El entrenamiento largo (E5c) da la mejora grande: 0.188 por encima de E4-A. Sus cambios no se separaron. E5d pasa de 0.175 a 0.625 con la única diferencia de la activación. Las ReLU muertas causaron el colapso.

Análisis del mejor modelo

El mejor grupo por validación es E5c, y dentro de E5c, la semilla 1 (0.900 de validación).

- **Curvas.** Hasta la época 45, la tasa todavía es alta y las curvas son inestables. Después, el decaimiento coseno estabiliza todo.
- **Test.** 66 de 80 aciertos: 0.825, con un intervalo de Wilson de 95% de [0.727, 0.893]. Frente a E1 con la misma semilla, la diferencia pareada es +0.2875, con un intervalo bootstrap de [+0.1625, +0.4125]. La caída desde 0.900 en validación es esperable al elegir un máximo.

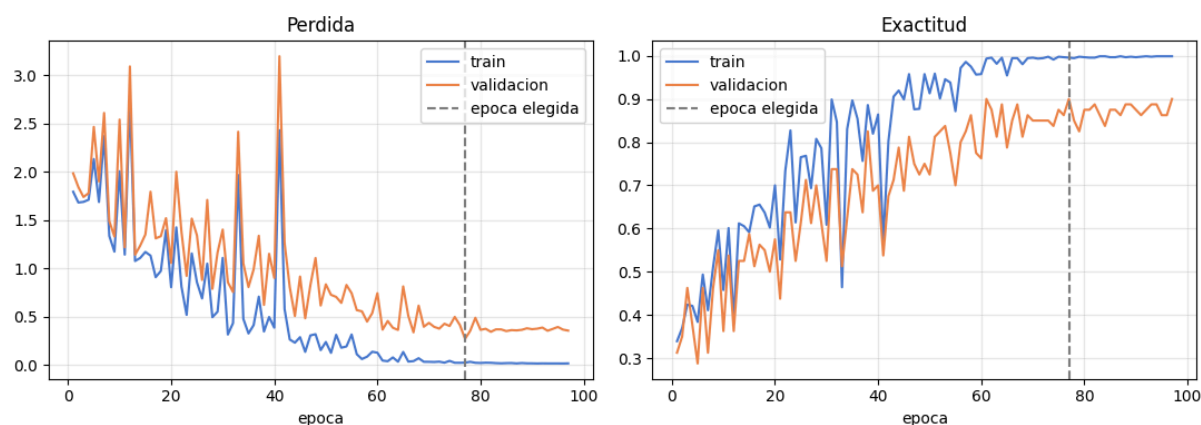


Figura 12. Curvas de pérdida y exactitud del mejor modelo (E5c, semilla 1).

Qué razas se confunden. Se esperaban errores dentro del trío de Collies y del par Labrador-Golden (predicción P6). La respuesta depende del modelo.

- **Con E4-A, P6 falló.** 20 de los 25 errores predecían Siberian Husky (11) o Labrador (9). Eran perros de pelo claro o con pecho blanco. La hipótesis es que el modelo usa más el color del pelo que la forma. Esta hipótesis no se probó.
- **Con E5c, P6 se cumple en parte.** Los dos grupos son el 14.3% de las celdas, y reciben el 29% de los errores. Pero 10 de los 14 siguen fuera de los grupos.

- **La atracción hacia Husky baja.** Con E5c, solo 3 errores predicen Husky, todos de Collie. Un Husky de cara blanca y negra se predice como Border Collie. Es coherente con la hipótesis del color.
- **Por raza,** E5c acierta de 6 a 10 imágenes de 10. No se afirma un orden entre razas, porque los intervalos son anchos.

raza real	Border Collie	10	0	0	0	0	0	0	
	Collie	0	6	1	0	0	0	3	
	Bearded Collie	0	0	10	0	0	0	0	
	Labrador	0	0	0	8	2	0	0	
	Golden Retriever	0	0	1	1	7	0	0	1
	Pug	0	0	0	0	0	10	0	0
	Siberian Husky	1	0	0	1	0	0	7	1
	Chihuahua	0	0	0	1	0	1	0	8
		raza predicha							
	Border Collie	Collie	Bearded Collie	Labrador	Golden Retriever	Pug	Siberian Husky	Chihuahua	

Figura 13. Matriz de confusión del mejor modelo (E5c) en test.

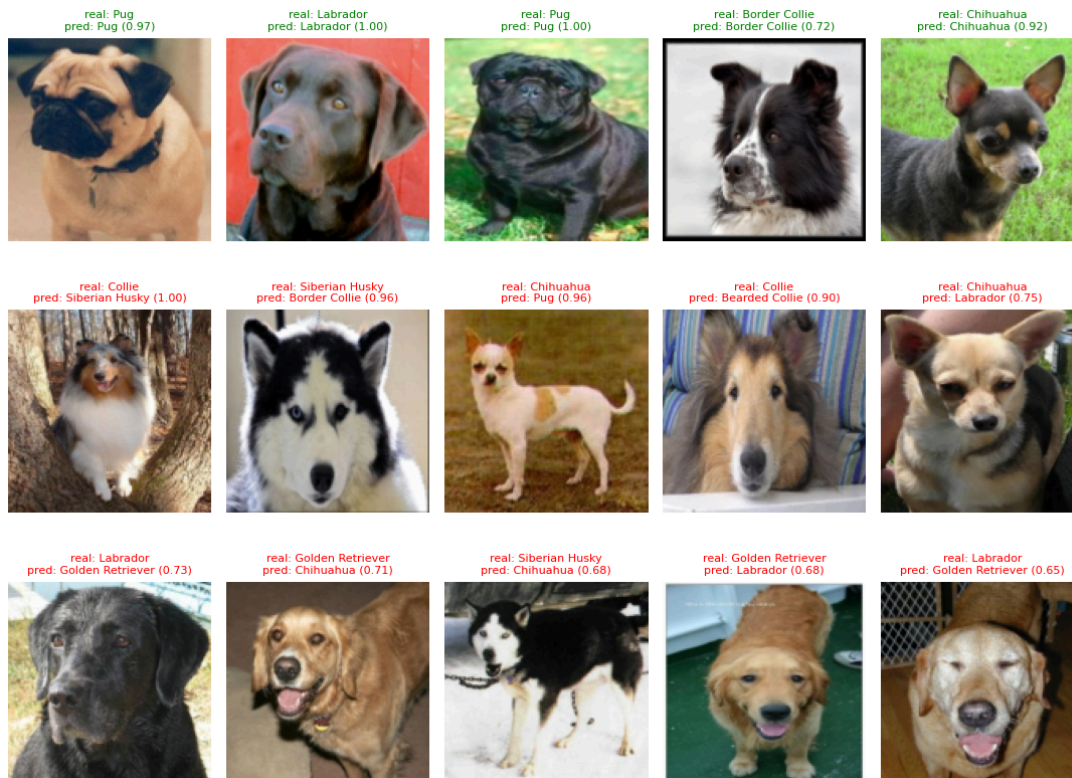


Figura 14. Cinco aciertos al azar (arriba) y los diez errores con mayor confianza del mejor modelo (E5c).

Un error que resume el problema de E4-A. Este Chihuahua tiene pelo gris claro, ojos grandes y orejas paradas. E4-A lo llama Siberian Husky, con confianza 0.82. Un Husky pesa unas 10 veces más. Pero la imagen

no tiene información de tamaño: un Chihuahua de cerca y un Husky de lejos ocupan el mismo cuadro de 128 x 128. E5c ya no comete este error.

real: Chihuahua
pred: Siberian Husky (0.82)

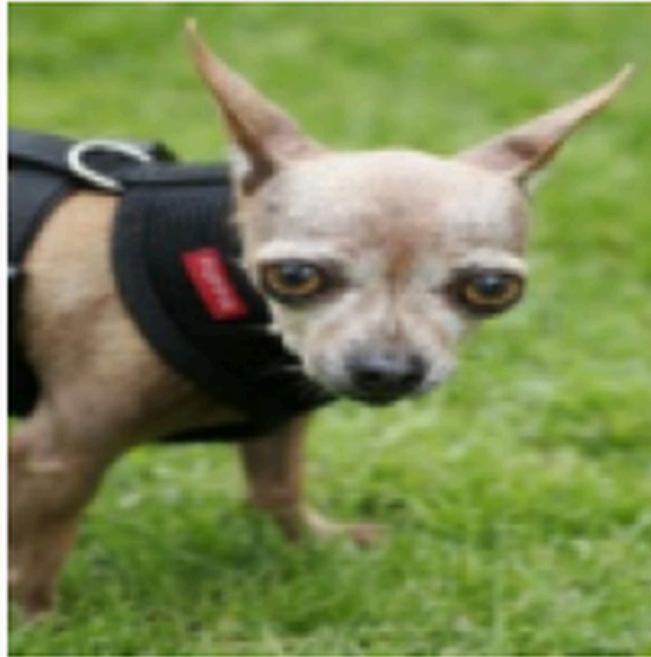


Figura 15. Un Chihuahua que E4-A llama Siberian Husky, con confianza 0.82.

Desafíos encontrados

- **Sin torchvision.** La augmentation se escribió con tensores de PyTorch.
- **La reanudación no repetía el resultado.** El checkpoint no guardaba el generador global de PyTorch, que usa el Dropout. Con ese estado en el checkpoint, la diferencia es 0.
- **Un error en la augmentation.** La traslación movía la imagen la mitad de lo documentado, porque `affine_grid` usa coordenadas de -1 a 1. Se corrigió el factor y se agregaron pruebas de magnitud.
- **El test ya se había visto cuando se diseñó E5.** E5c se eligió solo con validación, y se declara que su test es algo optimista.
- **Reproducibilidad en GPU.** Las opciones deterministas de PyTorch dan diferencia 0 también en la GPU.

Conclusiones

- **Con 920 imágenes, la arquitectura no cambió la exactitud.** Las cinco arquitecturas de E1 a E3 quedan dentro del ruido entre semillas.
- **Lo que más importó fue el entrenamiento.** La data augmentation sube la validación de 0.525 a 0.683. El entrenamiento largo con decaimiento coseno, sobre la cabeza GAP, la sube a 0.871. El mejor modelo (E5c) acierta 0.825 en test [0.727, 0.893], frente a un azar de 0.125.
- **El Dropout falló por las ReLU muertas, y esto se probó.** Sin el diagnóstico, la conclusión era que el Dropout no sirve. La medición dio una causa concreta, y la causa dio la corrección.
- **Los errores cambian con el modelo.** E5c tiene 14 errores en lugar de 25, y más de ellos caen entre razas parecidas.
- **Limitaciones.** Validación y test tienen 80 imágenes. Se usaron 3 semillas y una sola división. Las partes de E5c no se separaron. Quedan sin probar un bloque Inception, una resolución de 224 y Grad-CAM para la hipótesis del color.